

WEEK 5 DBMS PROJECT REPORT

FOOD ORDERING SYSTEM

Payment Management Module

Database Management System

1. INTRODUCTION

A Food Ordering System manages information related to customers, food orders, and payments. A reliable payment module is important because it records how an order was paid and whether the transaction was completed, failed, or is pending. The supplied Week 5 material is centered on payment queries, including transaction retrieval, payment-status filtering, payment-mode counts, revenue calculation, and a combined customer-order-payment report.

2. OBJECTIVES

- To maintain payment details for food orders.
- To retrieve all payment records from the Payment table.
- To display completed and failed payment transactions.
- To count transactions based on payment mode.
- To calculate the total revenue from completed payments.
- To summarize completed revenue by payment mode.
- To combine customer, order, and payment information in a single report.
- To practice SQL commands and database relationships for a Food Ordering System.

3. SYSTEM OVERVIEW

The Food Ordering System can be organized around three core entities used by the Week 5 payment work: CUSTOMER, ORDERS, and PAYMENT. A customer places an order, and the order can have an associated payment record. The Payment table stores the payment identifier, order identifier, payment date, payment mode, payment status, and transaction amount.

4. DATABASE TABLES AND RELATIONSHIPS

Table	Purpose	Relationship
CUSTOMER	Stores customer information.	Customer places orders.
ORDERS	Stores food order information.	Linked to CUSTOMER and PAYMENT.
PAYMENT	Stores payment transaction details.	Linked to an order using ORDER_ID.

5. PAYMENT TABLE STRUCTURE

Column Name	Data Type	Description
PAYMENT_ID	VARCHAR2(7)	Unique payment identifier.
ORDER_ID	VARCHAR2(7)	Identifier of the related food order.
PAYMENT_DATE	DATE	Date on which payment was recorded.
PAYMENT_MODE	VARCHAR2(20)	Payment method such as UPI, CARD, CASH, or NET BANKING.
PAYMENT_STATUS	VARCHAR2(10)	Transaction state such as COMPLETED, FAILED, or PENDING.
TRANSACTION_AMOUNT	NUMBER(10,2)	Amount of the payment transaction.

```
SQL> CREATE TABLE PAYMENT(PAYMENT_ID VARCHAR(7), ORDER_ID VARCHAR(7), PAYMENT_DATE DATE, PAYMENT_MODE VARCHAR(20), PAYMENT_STATUS VARCHAR(10), TRANSACTION_AMOUNT NUMBER(10,2));
```

Table created.

```
SQL> DESC PAYMENT;
```

Name	Null?	Type
PAYMENT_ID	NOT NULL	VARCHAR2(7)
ORDER_ID	NOT NULL	VARCHAR2(7)
PAYMENT_DATE	NOT NULL	DATE
PAYMENT_MODE	NOT NULL	VARCHAR2(20)
PAYMENT_STATUS	NOT NULL	VARCHAR2(10)
TRANSACTION_AMOUNT	NOT NULL	NUMBER(10,2)

SQL QUERIES, CONSTRAINT AND OPERATIONS

PRIMARY KEY:

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT pk_PAYMENT PRIMARY KEY (PAYMENT_ID);  
Table altered.
```

FOREGIN KEY:

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT fk_PAYMENT_ORDER FOREIGN KEY (ORDER_ID) REFERENCES ORDERS(ORDER_ID);  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_PAYMENT_MODE CHECK (PAYMENT_MODE IN ('UPI', 'CARD', 'NET BANKING', 'CASH'));  
Table altered.  
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_PAYMENT_STATUS CHECK (PAYMENT_STATUS IN ('COMPLETED', 'FAILED', 'PENDING'));  
Table altered.  
SQL> ALTER TABLE PAYMENT ADD CONSTRAINT chk_TRANSACTION_AMOUNT CHECK (TRANSACTION_AMOUNT >0);
```

NOT NULL:

```
SQL> ALTER TABLE PAYMENT MODIFY (ORDER_ID NOT NULL, PAYMENT_DATE NOT NULL, PAYMENT_MODE NOT NULL, PAYMENT_STATUS NOT NULL, TRANSACTION_AMOUNT NOT NULL);  
Table altered.
```

VALUE INSERTED IN PAYMENT TABLE:

```
SQL> INSERT INTO PAYMENT  
2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)  
3 VALUES  
4 ('PAY101', '106', TO_DATE('2026-09-24','YYYY-MM-DD'), 'UPI', 'COMPLETED', 3000);
```

1 row created.

```
SQL> INSERT INTO PAYMENT  
2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)  
3 VALUES  
4 ('PAY102', '107', TO_DATE('2026-05-15','YYYY-MM-DD'), 'CASH', 'PENDING', 900);
```

1 row created.

```
SQL> INSERT INTO PAYMENT  
2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)  
3 VALUES  
4 ('PAY103', '108', TO_DATE('2026-03-09','YYYY-MM-DD'), 'NET BANKING', 'FAILED', 1000);
```

1 row created.

```
SQL> INSERT INTO PAYMENT  
2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)  
3 VALUES  
4 ('PAY104', '109', TO_DATE('2026-05-10','YYYY-MM-DD'), 'CARD', 'PENDING', 2000);
```

1 row created.

```
SQL> INSERT INTO PAYMENT  
2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)  
3 VALUES  
4 ('PAY105', '110', TO_DATE('2026-10-11','YYYY-MM-DD'), 'NET BANKING', 'COMPLETED', 1500);
```

1 row created.

Display All Payment Records

Purpose: To display all records stored in the PAYMENT table.

```
SELECT * FROM PAYMENT;
```

```
SQL> SELECT * FROM Payment;
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY101	106	24-SEP-26	UPI	COMPLETED	3000
PAY102	107	15-MAY-26	CASH	PENDING	900
PAY103	108	09-MAR-26	NET BANKING	FAILED	1000
PAY104	109	10-MAY-26	CARD	PENDING	2000
PAY105	110	11-OCT-26	NET BANKING	COMPLETED	1500

6.2 Display Completed Payments

Purpose: To display only successfully completed payment transactions.

```
SELECT * FROM PAYMENT  
WHERE PAYMENT_STATUS = 'COMPLETED';
```

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED';
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY101	106	24-SEP-26	UPI	COMPLETED	3000
PAY105	110	11-OCT-26	NET BANKING	COMPLETED	1500

6.3 Display Failed Payments

Purpose: To retrieve payment transactions whose status is FAILED.

```
SELECT * FROM PAYMENT  
WHERE PAYMENT_STATUS = 'FAILED';
```

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'FAILED';
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PAY103	108	09-MAR-26	NET BANKING	FAILED	1000

COUNT FOR PAYMENT_STATUS:

```
SQL> SELECT PAYMENT_STATUS, COUNT(PAYMENT_STATUS) FROM PAYMENT GROUP BY PAYMENT_STATUS;
```

PAYMENT_ST	COUNT(PAYMENT_STATUS)
PENDING	2
FAILED	1
COMPLETED	2

6.4 Count UPI Transactions

Purpose: To count the number of payment transactions made through UPI.

```
SELECT COUNT(*) AS UPI_TRANSACTIONS  
FROM PAYMENT  
WHERE PAYMENT_MODE = 'UPI';
```

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE = 'UPI';
```

UPI_TRANSACTIONS
1

6.5 Count Card Payments

Purpose: To count card-based payments.

```
SELECT COUNT(*) AS CARD_PAYMENTS  
FROM PAYMENT  
WHERE PAYMENT_MODE IN ('CREDIT CARD','DEBIT CARD');
```

```
SQL> SELECT COUNT (*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE = 'CARD';
```

CARD_PAYMENTS
1

6.6 Calculate Total Revenue from Completed Payments

Purpose: To calculate the total transaction amount for completed payments.

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED';
```

```
TOTAL_REVENUE
```

```
-----  
4500
```

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

```
PAYMENT_MODE    TOTAL_REVENUE
```

```
-----  
UPI              3000  
NET BANKING      1500
```

6.7 Revenue by Payment Mode

Purpose: To calculate completed-payment revenue for each payment mode and arrange the result from highest to lowest revenue.

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;
```

```
PAYMENT_MODE    TRANSACTION_COUNT
```

```
-----  
NET BANKING      2  
CARD              1  
UPI              1  
CASH              1
```

6.8 Customer, Order and Payment Details

Purpose: To combine customer, order, and payment information so that the payment details of food orders can be viewed together.

```
SELECT C.CUSTOMER_NAME,
       O.ORDER_ID,
       O.ORDER_DATE,
       P.PAYMENT_DATE,
       P.PAYMENT_MODE,
       P.TRANSACTION_AMOUNT AS AMOUNT,
       P.PAYMENT_STATUS
FROM CUSTOMER C
JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID
ORDER BY P.PAYMENT_DATE;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PAYMENT_DATE	PAYMENT_MODE	AMOUNT	PAYMENT_STATUS
ANITHA	108	09-MAR-26	09-MAR-26	NET BANKING	1000.00	FAILED
SURESH	109	10-MAY-26	10-MAY-26	CARD	2000.00	PENDING
VIJAY KUMAR	107	15-MAY-26	15-MAY-26	CASH	900.00	PENDING
SNEHA	106	23-SEP-26	24-SEP-26	UPI	3000.00	COMPLETED
KAMESH	110	10-OCT-26	11-OCT-26	NET BANKING	1500.00	COMPLETED

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION FROM
PAYMENT WHERE PAYMENT_STATUS = 'COMPLETED' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
UPI	1	3000	3000
NET BANKING	1	1500	1500

8. RESULT

The Payment Management Module for the Food Ordering System was implemented using SQL. The queries can retrieve payment records, filter transactions by status, count payment modes, calculate completed-payment revenue, group revenue by payment mode, and generate a combined customer-order-payment report.

9. CONCLUSION

The Week 5 project demonstrates how SQL can be used to manage and analyze payment information in a Food Ordering System. By connecting customer, order, and payment tables, the system provides useful information for transaction monitoring and revenue analysis. The project also provides practical experience with SELECT, WHERE, COUNT, SUM, GROUP BY, ORDER BY, and JOIN operations.

10. FUTURE ENHANCEMENTS

- Add a FOOD or MENU table to store food items and prices.
- Add an ORDER_ITEM table to store multiple food items within each order.
- Add delivery-status and delivery-address information.
- Add discount, tax, and final-bill calculations.
- Add payment validation and refund tracking.
- Create reports for daily, weekly, and monthly food-order revenue.